

3D-TokSIM: Stacking 3D Memory with Token-Stationary Compute-in-Memory for Speculative LLM Inference

Wentao Zhao¹, Boya Lv¹, Meng Wu¹, Peiyu Chen^{2,3}, Fengyun Yan², Yufei Ma¹,
Tianyu Jia^{1,*}, Ru Huang¹, and Le Ye^{1,3,*}

¹Peking University, Beijing, China

²Nano Core Chip Electronic Technology, Hangzhou, China

³Advanced Institute of Information Technology of Peking University, Hangzhou, China

*Corresponding Authors: tianyu.jia@pku.edu.cn, yele@pku.edu.cn

Abstract—The LLM decoding process poses a significant challenge for memory bandwidth due to its autoregressive nature. Prior 2D memory solutions fail to overcome this memory bottleneck due to limited memory-to-logic bandwidth. In this work, we propose 3D-TokSIM, a cross-stack solution by stacking 3D memory on logic die with a specially designed token-stationary compute-in-memory (CIM) to efficiently accelerate speculative decoding. Our CIM is developed with novel token-stationary dataflow to reduce data movement on logic die to save power and balance computation and memory access. To further reduce the buffer requirements, we perform architecture exploration and allocate notable CIM resources for achieving higher decoding parallelism. Compared to RTX 3090 GPU, 3D-TokSIM achieves 15.1× throughput and 324× energy efficiency improvements on speculative Llama2-7B decoding.

Index Terms—LLM, Process-Near-Memory, Compute-In-Memory

I. INTRODUCTION

Large Language Models (LLMs) have demonstrated remarkable capabilities in natural language processing tasks and beyond. However, it is quite challenging for existing hardware solutions, e.g. GPUs, to achieve efficient LLM deployment. Due to the intrinsic autoregressive model nature, LLMs require loading the entire model to generate one new token at each decoding step [1]. Such operation behaviors lead to a severe bottleneck of memory bandwidth for LLM inference. Meanwhile, the existing GPUs or XPU are designed with massively parallel computing units with quite limited off-chip memory bandwidth. Hence it is clear from many research papers that the LLM inference is a memory-bound task and desires cross-stack optimizations for bandwidth [2]–[5].

To alleviate the bandwidth bottleneck, 3D memory architecture using the hybrid bonding (HB) technique is a promising solution [6], [7]. By stacking the DRAM memory on top of the compute logic die, DRAM bandwidth can be significantly improved by parallel accessing DRAM banks, which were originally accessed at interleaved time. Based on the public data, the HB 3D memory can provide 135× bandwidth and 8× density improvements with more than 90% energy saving compared to existing 2D DRAM [8], [9].

Although 3D memory stacking brings valuable memory bandwidth, there is little exploration on how to efficiently deploy LLMs on such architecture. Even with 3D technology, the entire architecture remains memory-bound due to the inefficient autoregressive model behavior. A promising algorithmic strategy to alleviate the bandwidth bottleneck is speculative decoding (SD) technique [4], which utilizes an efficient drafter to produce multiple tokens and then simultaneously sends these tokens into LLM model for parallel verification.

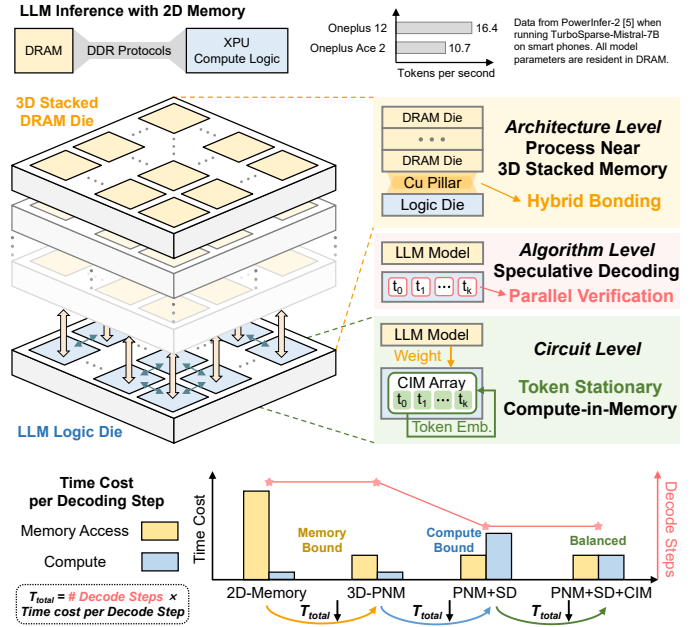


Fig. 1. Efficient LLM deployment on 3D stacking technique with cross-stack optimizations.

SD can successfully reduce the total decoding steps and alleviate the bandwidth requirement, with the cost of increasing computational workload in each decoding step. However, conventional digital design approach fails to efficiently process SD at the logic die, which effectively nullifies the high-bandwidth advantage of 3D memory stacking.

In this work, we explore a LLM deployment solution **3D-TokSIM**, which combines **3D** stacked DRAM processing-near-memory (PNM) and SRAM-based **token stationary** compute-in-memory (CIM) for efficient speculative inference. As shown in Fig.1, at the architecture level, we leverage 3D stacked memory to significantly enhance the memory access bandwidth for each decoding step. At the algorithm level, we leverage the latest speculative decoding approach for the 3D stacked architecture to further reduce the number of decoding steps and bandwidth requirements. At the circuit level, we employ compute-in-memory (CIM) design with token stationary dataflow to efficiently support SD. Our cross-stack design and optimizations achieve fewer decoding steps with a balanced memory-compute workload per step. This results in reduced decoding time and a higher rate of token generation. Evaluated on our design using TSMC 22nm, it is observed that 3D-TokSIM achieves 15.1× throughput

and $324\times$ energy efficiency improvements compared to RTX 3090 GPU. Moreover, 3D-TokSIM outperforms systolic array accelerated PNM of both throughput and energy efficiency by $1.5\times$ and $6.4\times$, respectively. To the best of our knowledge, this is the first exploration work that enables fast LLM inference using 3D stacking on top of CIM architecture.

The contributions of this paper are summarized as follows:

- We propose 3D-TokSIM, a 3D stacked DRAM and SRAM-based CIM architecture targeting speculative LLM inference.
- We leverage token stationary dataflow with careful optimizations to balance the CIM design and DRAM bandwidth.
- We further perform several buffer optimizations to place abundant CIM for higher decoding parallelism.
- Our evaluation demonstrates that the proposed 3D-TokSIM has $15.1\times$ throughput and $324\times$ energy efficiency improvements compared to RTX 3090 GPU.

II. BACKGROUND

A. Hybrid Bonding 3D Stacking

The ever-growing size of AI models has significantly exceeded the capacity of on-chip SRAM, resulting in frequent data transfers between DRAM and the main processor chip. Hybrid bonding process-near-memory (HB-PNM) [7] has emerged as a promising solution to the “memory wall” problem, offering fast and energy-efficient data access. In this approach, DRAM and logic die are integrated through direct copper-to-copper bonding. Each logic block can directly access its closely coupled DRAM banks and communicate with other logic blocks via the on-chip network (NoC). HB-PNM can achieve ultra-high interconnection density with a $3\mu\text{m}$ pitch compared to about $50\mu\text{m}$ microbump (ubump) pitch in HBM. Moreover, the resistance of HB is also much lower than ubump and through-silicon-vias (TSVs) [6].

HB technique has been initially studied for advanced memories. For example, SEDRAM [6] connected a DRAM array wafer and logic wafer using HB technology, with control, I/O, and peripherals located on the logic die. This motivates HB-PNM solutions for memory intensive applications. Niu et al. [7] developed an HB-PNM engine for recommendation systems that significantly outperformed CPU-DRAM system efficiency. Yue et al. [10] accelerated vision AI models with HB-PNM and gained notable efficiency improvement compared to prior accelerators. In addition to single layer DRAM stacking, multilayer array stacking is achieved with HB and mini-TSV in [9]. Driven by this, Chen et al. [11] presented an encryption engine for private inference with 4-layer stacked DRAM.

In the era of LLMs, it has already been well-studied that LLM decoding is heavily constrained by DRAM bandwidth [2], [3]. The memory bandwidth bottleneck is more severe for mobile devices, such as mobile phones or laptops. For example, mobile devices with the LPDDR5x protocol generally offer a peak bandwidth of 68GB/s , e.g. SK hynix 24GB LPDDR5x mobile DRAM [12]. When running a 4-bit quantized Llama-7B model, the theoretical throughput upper bound is 19.4 tokens per second (TPS), estimated by dividing the model size by the peak bandwidth. After adopting HB-PNM for edge devices, TBps-level bandwidth can be obtained resulting in approximately 500 TPS and $23\text{-}29\times$ bandwidth improvement compared to standard LPDDR5x.

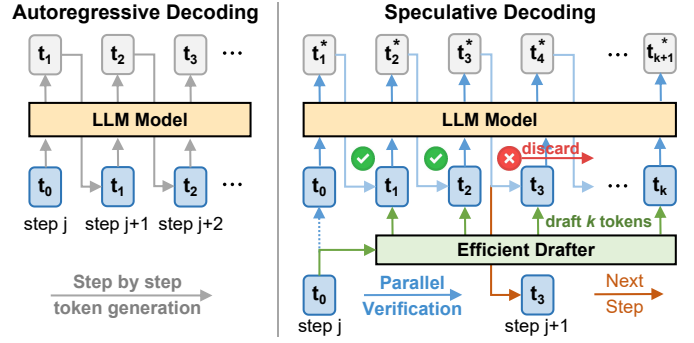


Fig. 2. Autoregressive decoding vs. Speculative decoding.

B. Speculative Decoding

Despite the use of high-bandwidth 3D technology, the LLM decoding may still be bounded by memory access due to its inherently autoregressive nature. Based on previous profiling [3], when the OPT-6.7B model is executed on an A100 GPU, the GEMV operation during the decoding phase accounts for over 80% of the total latency, while the GPU utilization remains below 25%. Speculative decoding [4] has emerged as a promising paradigm to boost LLM decoding by enabling the verification of multiple tokens in a single decoding step.

As shown in Fig. 2, SD first employs an efficient drafter to generate multiple tokens and then sends these tokens into the model in parallel to verify if they align with the LLM inference results. For a sequence of tokens to be verified, the first token that does not match the LLM result should be replaced with the LLM output, and any subsequent tokens should be discarded [4]. An important hyperparameter for SD is the number of draft tokens, denoted as k . While a higher k can lead to fewer decoding steps, the computational overhead grows up with k , potentially causing two other problems: 1) memory-bounded decoding steps will be shifted to computation-bounded, and 2) computations for those unaccepted tokens are inefficient and waste energy. Hence it is crucial to co-optimize SD across algorithm and hardware for optimal efficiency.

Since the first presentation of SD, there have been quite a few algorithm variants of SD to reduce the bandwidth requirement. For example, Leviathan et al. [13] utilized the small version of the T5 series to accelerate the larger ones. Yang et al. [14] introduced an early exiting scheme to initiate future tokens drafting, and He et al. [15] retrieved relevant tokens from a knowledge reservoir to draft tokens. Among these variants of SD, Inference with Reference (IwR) [16] is highly efficient and easy to deploy since it does not require an additional model for drafting. By exploring the massive overlaps between in-context references and outputs, IwR directly copies tokens from reference for parallel verification. This approach is highly practical in scenarios such as retrieval-augmented generation, cache-assisted generation, and multi-turn conversations.

III. 3D-TOKSIM ARCHITECTURE

A. Overall Architecture

In this paper, we propose **3D-TokSIM**, a **3D**-stacked architecture with **token stationary CIM** for efficient LLM inference. As shown in Fig. 3 left, 3D-TokSIM is composed of multiple parallel LLM

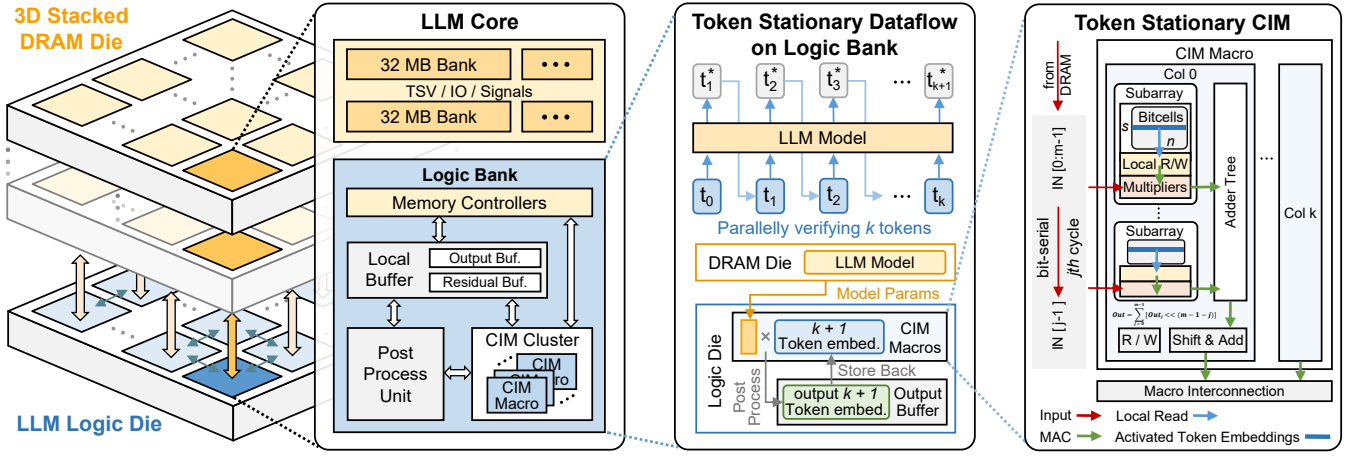


Fig. 3. Architecture overview: Stacking 3D memory above the logic die with token-stationary compute-in-memory clusters.

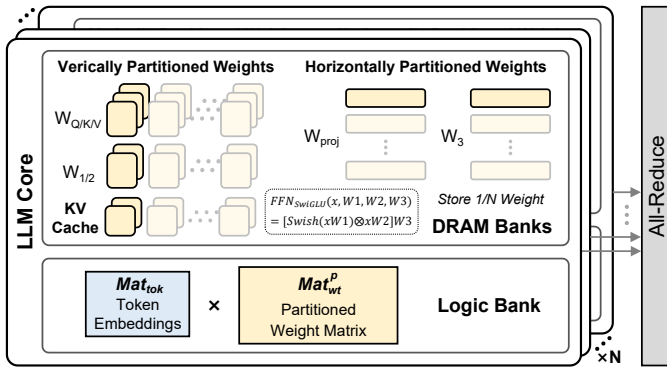


Fig. 4. Distributed Mapping for Parallel Acceleration.

Cores, and each core is composed of several DRAM Banks on DRAM dies and one logic bank on a logic die. DRAM dies and logic die are closely integrated through HB technique, which allows fast and energy-efficient DRAM access for logic banks. Each logic bank consists of memory controllers corresponding to above DRAM banks, a post-processing unit for handling non-linear operations, a CIM cluster responsible for multiply-and-accumulation (MAC) operation, and a local buffer. The LLM cores are connected through a NoC to enable inter-core data transfer.

Fig. 4 shows a typical mapping process to deploy multi-head-attention (MHA) and feed-forward-network (FFN) onto N parallel LLM Cores. For MHA, each LLM Core requires storing vertically partitioned weights for Q/K/V generation, KV cache, and horizontally partitioned projection weights. For FFN, the weights are partitioned vertically before activation and horizontally after activation. During LLM decoding, each logic bank loads weight or KV cache from its corresponding DRAM banks and performs multiplication with token embeddings. Following the last multiplication in MHA/FFN, an all-reduce operation merges results across cores, and then the full results are added to the initial token embeddings in the residual buffer to complete residual connection.

For each LLM core, we designate the matrix multiplications (MMs) in charge as $Mat_{tok} \times Mat_{wt}^p$, where p stands for partition. Either Mat_{wt}^p or Mat_{tok} can be mapped to the CIM cluster, which corresponds to **weight stationary** or **token stationary** dataflow.

Noticed that even though Mat_{wt}^p constitutes only $1/N$ of the original weight matrix, its size remains excessively large. Consequently, Mat_{wt}^p must be divided into smaller tiles when being mapped to CIM cluster, rendering two disadvantages: 1) CIM cluster needs to frequently update its weight, and Mat_{tok} must be read from local buffer repeatedly, resulting in massive data read/write operations, and 2) The input buffer is necessary to hold the entire input token embeddings, which further tightening the area constraints.

In contrast, we map Mat_{tok} instead of Mat_{wt}^p to the CIM cluster, referred to as **token stationary** dataflow, to liberate our design from the aforementioned concerns. Since the IwF typically dispatches a small group of tokens for parallel verification, it becomes possible to store the entire Mat_{tok} within the CIM by allocating sufficient storage resources. This eliminates the need for further tiling and input buffering.

B. Logic Die: Token Stationary CIM

In our work, a **token stationary** dataflow is developed to efficiently support speculative decoding. As illustrated in the middle part of Fig.3, the CIM macros hold all token embeddings Mat_{tok} in bitcell arrays. The DRAM sends model parameters as the input of CIM to perform MMs. The output result first undergoes post-processing and is held in the output buffer, which is then sent to CIM macros for subsequent MM.

Compared to conventional **weight stationary** dataflow, our **token stationary** dataflow reduces frequent read/write operations on the logic die and saves considerable energy. Note that due to low-resistance HB, the power consumption of data transfer from DRAM has been significantly reduced, for instance, to 0.88pJ/bit in [7] and 0.66pJ/bit in [9]. Consequently, the energy consumption of SRAM read/write operation is now comparable to that of DRAM data transfer. Therefore, we employ token stationary dataflow to eliminate frequent SRAM read/write operations. As shown in the middle of Fig.3, for every MM, we maintain all token embeddings Mat_{tok} in CIM and load weight from DRAM as CIM input. The entire MM process does not require storing weights in the CIM, nor does it require reading token embeddings from the local buffer. The outcomes are post-processed and stored back to CIM before subsequent MM.

TABLE I
NOTATIONS FOR DESIGN PARAMETERS AND LLM MODEL

Architecture Design Parameters	
N	Number of Parallel LLM Cores
$DRAM_{width}$	Number of DRAM I/O within a LLM Core
$DRAM_{freq}$	Frequency of DRAM I/O
CIM_{num}	Number of CIM Macros within a CIM Cluster
$Macro_{col}$	Number of CIM Columns within a CIM Macro
$Macro_{row}$	Number of Subarrays within a CIM Column
CIM_{freq}	Frequency of CIM
(s, n)	Rows and Columns of Subarray Bitcells
LLM Model Parameters	
dim	Model Dimension
d_{ff}	Intra-FFN Dimension
d_{head}	Attention Head Dimension
k	Number of Draft Tokens in Each Decoding Step

Our CIM macros are designed to meet the requirement of token stationary dataflow. As depicted in Fig.3 right, CIM macros are composed of several identical CIM columns. A CIM column has multiple CIM subarrays and an adder tree. Within each subarray, there are $s \times n$ SRAM bitcells, n local read/write (R/W) units, and n 1-b multipliers, where n equals the minimum supported data width. Token embeddings Mat_{tok} are stored within bitcells, which are then activated and read by local read/write units. The multipliers are shared by s row bitcells, where s indicates the optimal storage-compute ratio for token stationary dataflow. An m -bit input vector requires m cycles for bit-serial computation, and a shifter-and-adder is responsible for accumulating m partial sums. Multiple CIM macros can be connected vertically or horizontally to support various shapes of matrix multiplication. The notations utilized in this work for architecture design and LLM model are summarized in Table I.

For a single MM, each token embedding of length d performs a dot product with weights or KV cache, which is done in parallel for $k + 1$ token embeddings by broadcasting. Therefore, we store individual token embeddings in separate CIM columns, ensuring that each token embedding is mapped to its own distinct column:

$$Macro_{col} = k + 1, \quad n = TokenEmb_{width} \quad (1)$$

where $TokenEmb_{width}$ denotes the bit width of token embeddings.

Given that $k + 1$ is usually much smaller than d , a single CIM macro is designed to accommodate $k + 1$. Multiple macros are connected vertically to cover the entire length of the maximum d as follows:

$$CIM_{num} \times Macro_{row} \times s = \max(dim, \frac{d_{ff}}{N}) \quad (2)$$

IV. ARCHITECTURE OPTIMIZATIONS

A. Exploration Constraints

Based on the above analysis, we have identified token stationary as the suitable dataflow for speculative LLM inference. Furthermore, we perform architecture modeling to guide the design of token-stationary CIM cluster to take full advantage of this dataflow.

To fully exploit the 3D DRAM bandwidth, the CIM cluster should be equipped with sufficient computational resources. More specifically, the rate at which data are read from DRAM should be aligned with the rate at which CIM consumes them. The bit width

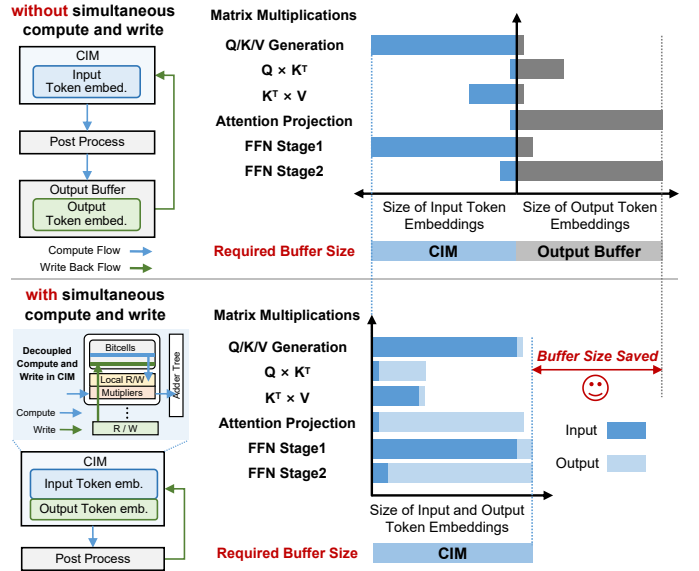


Fig. 5. Eliminating Buffer by Writing Output to CIM.

of data is represented by $Data_{bits}$. Within a unit of time T_{unit} , the number of data read from DRAM is represented by:

$$Data_{read} = T_{unit} \times \frac{DRAM_{width}}{Data_{bits}} \times DRAM_{freq} \quad (3)$$

Since data are sent into CIM bit-serially, the number of data consumed by CIM within T_{unit} is expressed as:

$$Data_{consume} = T_{unit} \times \frac{CIM_{num} \times Macro_{row} \times CIM_{freq}}{Data_{bits}} \quad (4)$$

Equalizing $Data_{read}$ and $Data_{consume}$ results in:

$$CIM_{num} \times Macro_{row} \times CIM_{freq} = DRAM_{width} \times DRAM_{freq} \quad (5)$$

However, the reduction length of intermediate token embeddings during MHA or FFN is small, and they should be copied to fully utilize CIM performance and DRAM bandwidth. For each copy, it should be able to perform MAC calculations independently. Therefore, the minimal reduction length achievable by CIM is set to correspond with the minimum d :

$$Macro_{row} = \min(d_{head}, \frac{d_{ff}}{N}) \quad (6)$$

Within the area constraints, the integer solution closest to the above requirements is selected for CIM design. For larger model dimensions, CIM capacity per column expands by adding more bitcells to subarrays. As CIM column equips more capacity or performance, its area increases, ultimately reducing the total number of CIM columns which corresponds to supported k .

B. Architecture Optimizations

Since the area of a single logic bank is constrained by the area of its corresponding DRAMs above it, the maximum supported k is also limited by the available CIM area. To ensure ample CIM resources, we further optimize the on-chip SRAM size and reduce the buffer requirement for each LLM core. Optimization opportunities are explored at both intra- and inter-core levels to minimize buffer requirements for MM output and residual connection, respectively.

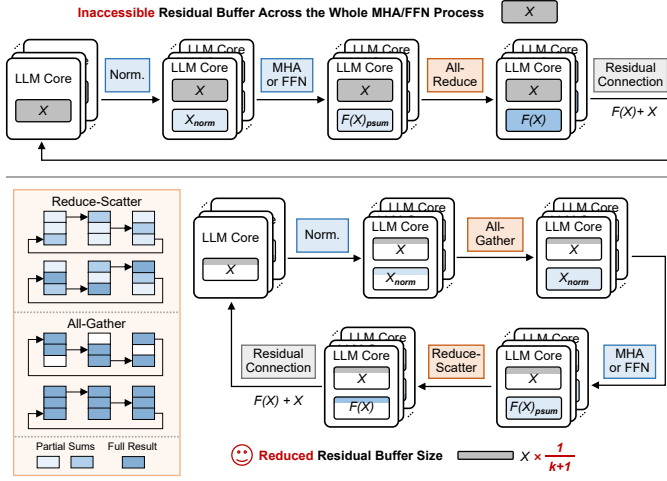


Fig. 6. Reducing Residual Buffer by Jointly Considering Residual Connection with All-Reduce Operation.

1) *Output Buffer Optimizations:* We first focus on optimizing the scheduling of MM within each core, aiming to eliminate output buffer. As illustrated in the Fig.5 top, our token stationary dataflow ensures input token embeddings are fully resident in CIM, while the output token embeddings are stored in the output buffer. The required capacity of CIM and output buffer are determined by the largest token embeddings size across all MMs as depicted in the upper right of Fig.5. Since the output of MM will be returned to CIM for subsequent MM, by enabling simultaneous compute and write of CIM, the output can be directly written back to CIM without temporarily residing in output buffer.

Therefore, we implement simultaneous compute and write CIM as shown in the lower left of Fig.5, where the local read/write units have decoupled multipliers and adder trees from bitcells, showing no resource contention between compute and write operation. Consequently, the elements of output token embeddings can be written back to CIM after being post-processed even if the CIM is still computing. As shown in the lower right of Fig.5, the output buffer is now eliminated at the cost of slightly increased CIM size, and we adjust our previous CIM design size in (2) to:

$$CIM_{num} \times Macro_{row} \times s = dim + \max(d_{head}, \frac{d_{ff}}{N}) \quad (7)$$

For the typical Llama2-7B model ($dim = 4096$, $d_{head} = 128$ and $d_{ff} = 11008$) accelerated with 32 cores, the capacity of CIM increases about 8.4% according to (7) and (2), which demonstrates minimal area overhead.

2) *Residual Buffer Optimizations:* Residual connection (RC, $F(X) + X$) impedes resource-constrained model inference as it requires extra buffer. Previous work has attempted to reorganize residual connections to enable tiny devices to accommodate CNN models [17]. As shown at the top of Fig.6, for parallel LLM inference, RC requires keeping the unnormalized input token embeddings of MHA or FFN, denoted as X , across the whole computing process. The buffer tailored for X is inaccessible during MHA or FFN, leading to constraint area for CIM.

As displayed at the bottom of Fig.6, we reduce the residual buffer size for each LLM core by jointly considering RC with all-reduce

TABLE II
3D-TOKSIM SPECIFICATION

System Configuration	
Number of Cores	32
Stacked DRAMs	4
DRAM within Single LLM Core	
Number of Banks	4
Bank Size	32 MB
Total Bandwidth	62.5 GB/s
CIM within Single LLM Core	
Number of Macros	8
Macro Size	128×13
Subarray Size	5×16
Performance	3.25 TOPS
Energy Efficiency	9.51 TOPS/W
Area	1.39 mm ²

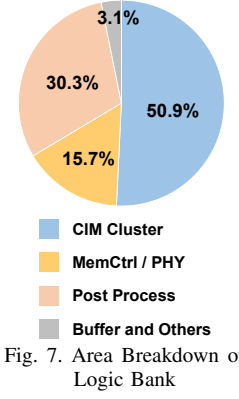


Fig. 7. Area Breakdown of Logic Bank

(AR) operation between cores. We adopt bandwidth optimal ring AR [18], [19], which is further divided into two phases: reduce-scatter (RS) and all-gather (AG). Noticing that each core holds only a portion of the full result after RS, inserting RC between RS and AG allows each core to store only the corresponding portion of X . In addition, AG begins after normalization and each core receives the normalized token embeddings from other cores. Because token embeddings are stored column-wise in CIM, vertical read/write mode similar to prior CIMs [20], [21] is leveraged to support above proposal with the aid of local read/write units. With k draft tokens in SD, our scheme reduces the residual buffer size in each core to $1/(k+1)$. For typical k larger than 5, the area cost of the residual buffer is reduced to less than 16.7%.

V. EVALUATION

A. Experiment Setup

Hardware Setup. In our work, we selected a 32MB DRAM bank with 256 I/O as the fundamental unit for our memory design, with area referring to [10]. Both DRAM die and Logic die are operating at 500MHz. The energy consumption of multilayer DRAM refers to the latest publication [9]. Our logic die is designed using TSMC 22nm technology. The area of memory controller and PHY are based on [11], which has similar technology node to ours. We designed an in-house 22nm CIM macro with post-layout simulation. The energy efficiency of our CIM macro is 9.51TOPS/W for W4A16/W8A8 data format with 50% input toggle rate and 50% weight bit-level sparsity. The local buffer is generated by Arm Memory Compiler. The area and power of processing unit are based on our RISC-V VPU with various special functions integration referred from [22]. A performance model incorporating above hardware information has been constructed for architecture exploration.

To fairly compare weight stationary (WS) and proposed token stationary (TS) dataflow, the CIM under WS is also carefully designed, where the storage and performance of CIM are set to ensure that the weight update and the computation overlap as much as possible. In addition, a systolic array based on Gemmini [23] is synthesized by Cadence Genus. It is used as a comparison baseline for more comprehensive architecture evaluation.

Algorithm Setup. We evaluated the speculative decoding using the model from [16]. For different draft lengths k ranging from 0 to 18, we recorded the decoding information across 100 samples. The decoding information includes the length of KV cache and whether

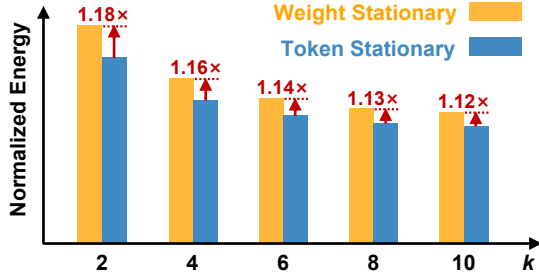


Fig. 8. Energy Cost of Weight Stationary and Token Stationary Dataflow.

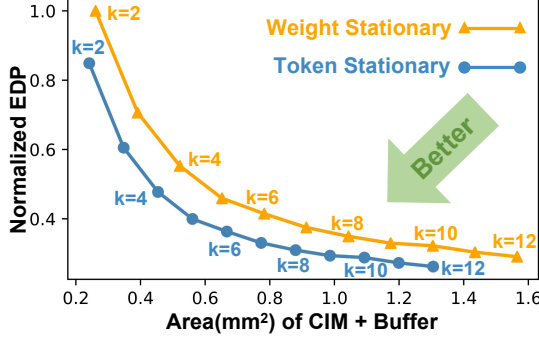


Fig. 9. EDP vs. Area of Weight Stationary and Token Stationary Dataflow.

coping k tokens or not in each decoding step. We use this record as the algorithm input for our performance model.

B. Implementation and Exploration

We implemented 3D-TokSIM with 32 LLM cores, with each core containing 4 DRAM banks stacked vertically to support the typical LLama2-7B [1] model with W4A16 quantization. The 4-layer stacked DRAM offers 2TB/s bandwidth and 4GB capacity in total. The specification of 3D-TokSIM is listed in Table II. Each LLM core contains 8 CIM macros, and the column of each CIM macro is 13 to support SD with k equals 12. These 8 cores together provide a performance of 3.25TOPS and 130KB capacity. The area breakdown of a single logic bank is depicted in Fig.7. It illustrates that our optimization ensures ample CIM area, while the buffer area is minimal.

We decomposed our architecture optimization into two parts: the energy saving through token stationary dataflow and the enhancements in energy-delay product (EDP) by further buffer reduction. Our optimization yields similar results across different models, and here we present the results for the 4-bit quantized LLAMA2-7B model. The energy costs of WS and TS dataflow with different k are shown in Fig.8. For k ranges from 2 to 10, the TS dataflow offers $1.18\times$ - $1.12\times$ better energy efficiency than WS. As k increases, the improvement decreases slightly. This is due to TS primarily optimizing data transfer overhead, while a larger k increases energy costs for computation. Fig.9 illustrates the EDP of WS and TS dataflows, with supported k ranging from 2 to 12. It can be clearly seen that TS always achieves better EDP than WS under the same area constraint. For k larger than 12, they are not depicted. This is because the EDP almost no longer decreases due to the higher compute overhead and saturated decoding steps.

Our architecture is extended to support various LLM models. Recently, low-bit quantization of LLMs has been developed, even

TABLE III
OPTIMIZATION OF VARIOUS LLM MODELS

LLM Model	Arch. Cfg.*	k Opt.	EDP Improve.
LLAMA2-7B-4bit	$32\times 1\times 4$	10 $\rightarrow$ 12	1.23 $\times$
LLAMA2-13B-2bit	$40\times 1\times 4$	9 $\rightarrow$ 12	1.21 $\times$
OPT-30B-3bit	$56\times 2\times 4$	11 $\rightarrow$ 12	1.13 $\times$
OPT-66B-2bit	$72\times 2\times 4$	10 $\rightarrow$ 12	1.16 $\times$

* Architecture Configuration: Number of LLM Cores $\times$ DRAM Banks per Core per Layer $\times$ Stacked DRAM Layers

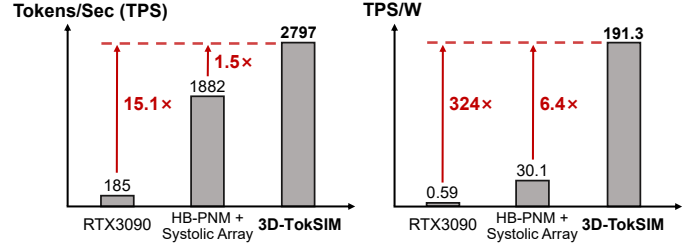


Fig. 10. Comparison result with RTX 3090 GPU and systolic array accelerated HB-PNM.

reaching down to 2-bit quantization [24]. Therefore different architecture configurations are set to adapt larger model sizes with 3/2 bit quantization. As listed in Table III, compared to WS dataflow, the optimized TS dataflow consistently supports larger k and brings $1.13\times$ - $1.23\times$ EDP improvement across various LLM models.

C. Comparison and Discussion

Fig.10 shows the comparison result of 3D-TokSIM with RTX 3090 GPU and systolic array accelerated HB-PNM on speculative Llama2-7B model. Our 3D-TokSIM generates 2797 tokens per second (TPS), which is $15.1\times$ faster than RTX 3090. Notably, the RTX 3090 provides a formidable 936GB/s DRAM bandwidth, and our 3D stacked memory only delivers twice the speed at 2TB/s. Therefore, this improvement is not solely due to higher bandwidth, but rather attributed more to the architecture. Compared to HB-PNM with systolic array acceleration, 3D-TokSIM achieves $1.5\times$ higher TPS. This is mainly due to the lower performance density of the systolic array compared to CIM, resulting in the system being compute-bound. In addition, the energy efficiency of 3D-TokSIM outperforms RTX 3090 and HB-PNM with systolic array acceleration by $324\times$ and $6.4\times$, which demonstrates the efficiency of proposed token stationary CIM.

VI. CONCLUSION

This paper presents a cross-stack solution for efficient LLM inference, i.e. 3D-TokSIM, which incorporates 3D stacked DRAM and token stationary CIM to efficiently accelerate speculative decoding. Optimization of the token stationary dataflow further reduces buffer needs, enhancing decoding parallelism and improving EDP with ample CIM resources. The implemented 3D-TokSIM using TSMC 22nm demonstrates $15.1\times$ and $1.5\times$ faster token generation speed with $324\times$ and $6.4\times$ energy efficiency compared to RTX 3090 and systolic array accelerated PNM.

ACKNOWLEDGMENT

This work was supported in part by the NSFC under Grant 62225401; and in part by the National Key R&D Program of China under Grant 2024YFB4505000.

REFERENCES

- [1] H. T. et al., “Llama 2: Open foundation and fine-tuned chat models,” 2023.
- [2] J. H. K. et al., “Samsung pim/pnm for transfrmer based ai : Energy efficiency on pim/pnm cluster,” in *2023 IEEE Hot Chips 35 Symposium (HCS)*, pp. 1–31, 2023.
- [3] S.-S. Park et al., “An lpddr-based cxl-pnm platform for tco-efficient inference of transformer-based large language models,” in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2024.
- [4] H. Xia et al., “Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding,” *arXiv preprint arXiv:2401.07851*, 2024.
- [5] Z. Xue et al., “Powerinfer-2: Fast large language model inference on a smartphone,” 2024.
- [6] B. Fujun et al., “A stacked embedded dram array for lpddr4/4x using hybrid bonding 3d integration with 34gb/s/1gb 0.88pj/b logic-to-memory interface,” in *2020 IEEE International Electron Devices Meeting (IEDM)*, 2020.
- [7] D. Niu et al., “184qps/w 64mb/mm23d logic-to-dram hybrid bonding with process-near-memory engine for recommendation system,” in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65, pp. 1–3, 2022.
- [8] K.-D. Hwang et al., “A 16gb/s/pin 8gb gddr6 dram with bandwidth extension techniques for high-speed applications,” in *2018 IEEE International Solid-State Circuits Conference - (ISSCC)*, pp. 210–212, 2018.
- [9] S. Wang et al., “A 135 gbps/gbit 0.66 pj/bit stacked embedded dram with multilayer arrays by fine pitch hybrid bonding and mini-tsv,” in *2023 IEEE Symposium on VLSI Technology and Circuits (VLSI)*, 2023.
- [10] Z. Yue et al., “Exploiting similarity opportunities of emerging vision ai models on hybrid bonding architecture,” in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pp. 396–409, 2024.
- [11] Z. Chen et al., “A high-throughput private inference engine based on 3d stacked memory,” in *61st ACM/IEEE Design Automation Conference (DAC)*, 2024.
- [12] Michail, “SK hynix is now shipping its 24GB LPDDR5X DRAM, will debut on OnePlus Ace 2 Pro.” https://www.gsmarena.com/sk_hynix_is_now_shipping_its_24gb_lpddr5x_dram_will_debut_on_oneplus_ace_2_pro-news-59516.php, 2023. [Online; accessed 2024-11-18].
- [13] Y. Leviathan, M. Kalman, and Y. Matias, “Fast inference from transformers via speculative decoding,” in *Proceedings of the 40th International Conference on Machine Learning, ICML’23, JMLR.org*, 2023.
- [14] S. Yang, G. Lee, J. Cho, D. Papailiopoulos, and K. Lee, “Predictive pipelined decoding: A compute-latency trade-off for exact llm decoding,” *ArXiv*, vol. abs/2307.05908, 2023.
- [15] Z. He, Z. Zhong, T. Cai, J. D. Lee, and D. He, “Rest: Retrieval-based speculative decoding,” *ArXiv*, vol. abs/2311.08252, 2023.
- [16] N. Yang et al., “Inference with reference: Lossless acceleration of large language models,” Tech. Rep. MSR-TR-2023-17, Microsoft, April 2023.
- [17] H. R. Mendis, C.-K. Kang, C.-H. Lin, M.-S. Chen, and P.-C. Hsiu, “Deep reorganization: Retaining residuals in tinymml,” in *Proceedings of the 61st ACM/IEEE Design Automation Conference, DAC ’24*, (New York, NY, USA), Association for Computing Machinery, 2024.
- [18] P. Patarasuk and X. Yuan, “Bandwidth optimal all-reduce algorithms for clusters of workstations,” *J. Parallel Distributed Comput.*, vol. 69, pp. 117–124, 2009.
- [19] Baidu, “baidu-research/tensorflow-allreduce.” <https://github.com/baidu-research/tensorflow-allreduce>. [Online; accessed 2024-10-09].
- [20] F. T. et al., “A 28nm 15.59µj/token full-digital bitline-transpose cim-based sparse transformer accelerator with pipeline/parallel reconfigurable modes,” in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65, pp. 466–468, 2022.
- [21] J. Wang et al., “A 28-nm compute sram with bit-serial logic/arithmetic operations for programmable in-memory vector computing,” *IEEE Journal of Solid-State Circuits*, vol. 55, no. 1, pp. 76–86, 2020.
- [22] E. Reggiani, R. Andri, and L. Cavigelli, “Flex-sfu: Accelerating dnn activation functions by non-uniform piecewise approximation,” *2023 60th ACM/IEEE Design Automation Conference (DAC)*, pp. 1–6, 2023.
- [23] H. G. et al., “Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration,” in *2021 58th ACM/IEEE Design Automation Conference (DAC)*, pp. 769–774, 2021.
- [24] J. Chee, Y. Cai, V. Kuleshov, and C. D. Sa, “Quip: 2-bit quantization of large language models with guarantees,” *Advances in neural information processing systems*, vol. 36, pp. 4396–4429, 2023.